→ (3.) Branch:- A branch is an independent line of development without disturbing main code.

`git switch -c feature-login` Now, we can work on the "feature-login" branch.

→ (4.) Remote:- A Remote is a repository stored somewhere else, usually on a service such as Github.

your computer
↓
Local Repository ⟷ Github ↓ Remote Repository (Push/Pull)

⊙ your local repository is on our local system.
⊙ The Remote repository is hosted online.

Eg:- `git push` → sends your commits from local → remote
`git pull` → get changes from remote → local.

⊛ Git Commands:- → (i.) git init - creates new repository in your current folder.

Before: my-project/
- index.html
- style.css
- app.js

After git init: my-project/
- index.html
- style.css
- app.js
- .git/

→ use this when you are starting Git tracking for your own existing project.

→ (ii.) git clone <url> :- Downloads an existing GIT repository to your computer.
It creates a local copy. use when there is existing repository.

→ (iii.) git status:- Shows the current state of your Git repository.
It tells: which files have changed
" " " untracked
" Branch you're currently on

Eg:- modified : app.py
untracked : login.py

<file>

→ (4.) git add <file> :- moves the changes of a specific file into the staging area.

`git add app.py` → you are telling git, I want to include the changes in app.py in my next commit.

→ (5.) git add . :- stages all changed/untracked files in the current directory.

Eg:- app.py ← Changed file
login.py ← Changed "
style.css ← Changed "

If we ren, `git add .` every file will go to staging.

Difference:- git add app.py → stages only app.py
git add . → stages all relevant changed files

→ (6.) git commit -m "message":- Saves the staged changes as a permanent checkpoint in Git history.

Eg:-
```
git add.
git commit -m "Added login page"
```

Similar checkpoints as; commit 1 → Added login page.
commit 2 → Added Registration.
commit 3 → Fixed login bug.

→ (7.) git log:- Shows the "commit history" of your repository.

`git log` → shows all saved check points.

Useful Version:- `git log --oneline`

o/p: a81f23a Added login page.
b72c11c Created Project.

→ (8.) git diff:- shows the changes you have made, but haven't committed yet.

Eg:- print("Hello")
Print("Hello world")

Run:- `git diff`

Git shows, what you have changed.

If we move changes to staging, like `git add .`

We might not see any thing for git diff

We should use `git diff --staged`

→ (9.) git restore <file>:- Discard uncommitted changes in a file (&) restore it to its previous committed state.

Eg: If we accidently change : app.py haven't committed, we use;

`git restore app.py` The changes are discarded.

→ (10.) git rm <file>:- Delete a file (&) tell Git about the deletion.

`git rm old_file.py` ① Deletes the file ② stages the deletion

`git commit -m "Removed: old file"`

→ (11.) git mv <old> <new>:- Rename (&) move a tracked file.

`git mv login.py user_login.py` Git understands that file was renamed.

(existing name → login.py, new name → user_login.py)

Then `git commit -m "Renamed login file"`

Branches → (12.) git branch:- show your branches `git branches`

Eg:- .git/ref/heads/ "*" means

Branches
- * main → still working on this branch (x)
- feature
- bugfix.

→ (13.) git branch <name>:- Create a new branch.

`git branch login-feature`

It creates, but it doesn't move it this, to work.

→ (14.) git switch <name> :- Moves you to another branch.

`git switch login-feature` git working on login-feature.

(15.) git switch -c <name>:- creates (&) immediately switches to a new branch.

(very common method)

`git switch -c payment-feature`

simply →
- git branch payment-feature.
- git branch payment-feature.

→ (16.) git merge <branch> :- If we have; main
commit A
commit B
login-feature
commit C
commit D.

you have finished the login feature.

First, go to main : git switch main

Next, merge : git merge login-feature

Git combines changes from login-feature into main.

simply, login-feature → merge → main

→ (17.) git branch -d <name> :- After merging a feature branch, we can delete it.

git branch -d login-feature

This removes the local branch, but it doesn't delete the commits that were merged into main.

Remote

repositories stored in github.

(18.) git remote -V :- Shows remote repositories connected to your local repository.

git remote -V

Eg:- origin https://github.com/..... (fetch)
origin https://..... (push).

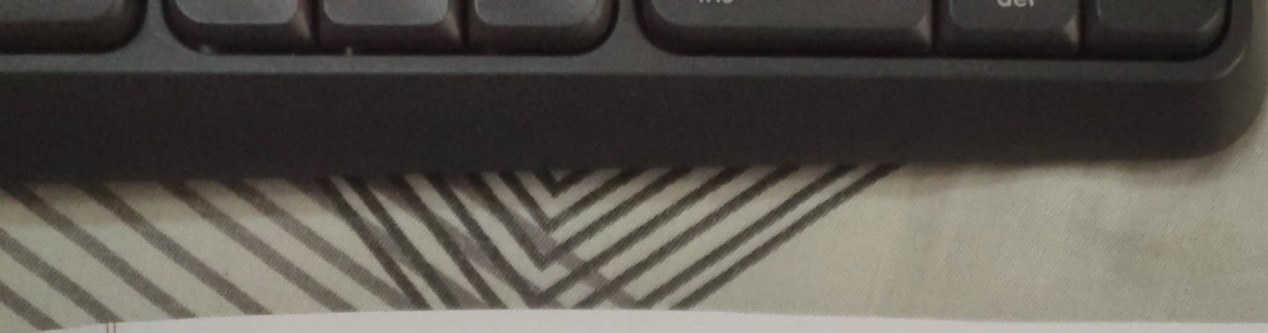
Origin ⇒ conventional name given to the remote repository.

(19.) git remote add origin <url> :- Connects your local repository to a git hub repository.

git remote add origin https://git.....

git remote -V } used to verify, the connection.

(20.) git push :- push (or) Send our local commits to the remote repository.



classmate
Date
Page

git push

Conceptually,
Local
↓ Push
GITHUB

Eg:-

```
git add .
git commit -m "Added login"
git push
```

(21) **git push -u origin main**:- Commonly used for the "first push" of a new local branch.

git push -u origin main
- -u → Set upstream tracking
- origin → name of the branch to push

After setting upstream, we can use **git push**

(22) **git pull**:- gets changes from the remote repository & integrate them into current branch (main branch).

git pull Eg: your teammate pushes some changes to GitHub, we run git pull, we get their changes locally.

(23) **git fetch**:- Download information about changes from remote repository without integrating those changes to our current branch.

git fetch	git pull
(1) Download remote changes	(1) Download & integrates changes
(2) Doesn't change current working branch	(2) Changes current working branch
(3) Safe for inspecting	(3) Convenient for getting updates

(*) git switch:

(*) git checkout :- Checkout is the older, multipurpose command.

For branches:- **git checkout main**. Switches to main.

git can also;

creates (&) switches to a branch:- **git checkout -b feature-login**

But, checkout can do other things too, such as restoring file (or) checking out a specific commit.

(*) git switch:- Introduced to make branch operations cleaner (&) safer. Only single feature - switching branch.

git switch main (switches to main).

git switch -c feature-login. (Create feature login (&) switching to it)

WORK FLOW:-

Create repository

↓

git init

↓

Local repo

↓

git remote add origin <URL>

↓

git status

↓

git add .

↓

git commit -m "___"

↓

git push -u origin main